

Survey Instrument and Design Rationale

PA2534 Master’s Thesis in Software Engineering

Decision-Making Under Uncertainty in Bug Triage and Defect Prioritisation

Naga Sai Abhiram Gopal Vemana Vineeth Kumar Vajja

Supervisor: Muhammad Laiq

May 2026

1 Methodological and Theoretical Design Rationale

Research Design: Convergent Mixed Methods. This study uses a *convergent mixed methods research design* [1–3], conducting qualitative semi-structured interviews and a quantitative cross-sectional survey concurrently in parallel. While the qualitative interviews provide in-depth explanations of how practitioners navigate complex triage decisions, the survey gathers broader cross-sectional data across different roles, team sizes, and software domains. Conducting both methods in parallel enables data triangulation [2], allowing us to compare qualitative findings with quantitative distributions to ensure the conclusions are grounded, dependable, and applicable across industry settings.

Theoretical and Empirical Context. The survey questions and response options are grounded directly in empirical software engineering research and cognitive decision-making models. Automated machine learning models are widely researched for bug assignment [4, 5], but they often struggle when defect data is incomplete or noisy [6]. In daily practice, bug reports frequently miss reproduction steps, system details, or clear descriptions [7–9]. When reports cannot be reproduced, triage becomes a human-driven decision process where engineers rely on experience, team communication, and practical rules of thumb [10–14].

Systematic Instrument Structure and Response Formats. The questionnaire is organized into six thematic parts to systematically capture operational bottlenecks, information deficiencies, cognitive heuristics, and organizational pressures:

- **Single-Choice Format:** Used for baseline questions (SQ1, SQ3, SQ4, SQ7, SQ8, SQ10) to identify the primary default practice or dominant bottleneck without data dilution.
- **Constrained Multi-Select Format:** Used where industry practice naturally involves multiple co-occurring factors (SQ2 for multiple tools, SQ6 for missing report details, and SQ9 for organizational pressures), constrained to “select up to two” to preserve statistical focus.
- **Dual-Response Format with Optional Open Text:** Pairing closed-ended questions with optional comment boxes (SQ4a, SQ6a, SQ7a, SQ8a, SQ9a, SQ10a) and narrative prompts (SQ5, SQ11) allows participants to describe unique workflows and edge cases [2, 15]. This generates rich qualitative quotes that directly triangulate with the interview data.

Target Population, Sampling, and Ethics. The survey targets active software practitioners with practical experience in software development, quality assurance, or defect triage. Participation is voluntary and completely anonymous. An upfront screening statement confirms industry background before participants begin the questionnaire. The survey is designed to take between 5 and 8 minutes to complete, maximizing completion rates and respecting participants’ time.

2 Survey Instrument

Introductory Statement and Informed Consent

This survey is part of a Master's thesis research study at Blekinge Institute of Technology (BTH) on Decision Making Under Uncertainty in Bug Triage and Defect Prioritisation, conducted by Naga Sai Abhiram Gopal Vemana and Vineeth Kumar Vajja under the supervision of Muhammad Laiq. Your responses are completely anonymous and will be used solely for academic research. The survey takes approximately 5 to 8 minutes to complete. By proceeding, you confirm that you have direct industry experience in software development or quality assurance, and consent to your anonymised responses being used in this study.

2.1 Participant Background and Context

Purpose: Collects basic demographic and contextual information regarding the respondent's primary role, tooling environment, and professional experience to support role-based comparisons across the survey sample [2].

SQ1 Which of the following best describes your primary role? *[Single Choice, Required]*

- ☐ Core Developer / Software Engineer
- ☐ Quality Assurance (QA) / Tester
- ☐ Project Manager / Scrum Master
- ☐ Dedicated Triage Lead / Bug Coordinator
- ☐ Other: _____

SQ2 Which issue-tracking systems do you primarily utilise? *[Multiple Choice, Required]*

- ☐ Jira
- ☐ Bugzilla
- ☐ GitHub Issues
- ☐ Proprietary / In-house Tool
- ☐ Other: _____

SQ3 How many years of professional experience do you have in software development and defect management? *[Single Choice, Required]*

- ☐ Less than 1 year
- ☐ 1 to 5 years
- ☐ 5 to 10 years
- ☐ More than 10 years

2.2 The General Triage Process

Purpose: Identifies the primary operational bottlenecks encountered during routine defect triage and quantifies where practitioners spend the majority of their triage time [4,10].

SQ4 What do you find to be the most time-consuming part of a typical bug triage session? *[Single Choice, Required]*

- ☐ Understanding the reporter’s description and verifying if it is a real bug.
- ☐ Attempting to reproduce the issue locally.
- ☐ Figuring out which developer has the right expertise to fix the bug.
- ☐ Negotiating the priority and severity with management or other teams.
- ☐ Other: _____

SQ4a *Optional: Is there another bottleneck not listed above, or any additional detail you would like to add to your answer?* *[Open Text Field]*

2.3 Sources of Uncertainty (O1 / RQ1)

Purpose: Directly addresses **Research Objective 1 (O1)** and **Research Question 1 (RQ1)**. Identifies the most common missing pieces of information in vague or non-reproducible bug reports, and collects real-world examples of difficult triage situations [7–9].

SQ5 Could you describe a specific situation you encountered in which a bug report was incomplete, ambiguous, or difficult to reproduce? *Please explain what information was missing or unclear, what you did next, and how the situation was eventually resolved, if it was.* *[Open Text Narrative]*

SQ6 When a bug is marked as “Non-Reproducible” or “WorksForMe,” what are usually the most common missing pieces of information? *[Select up to two options, Required]*

- ☐ Exact steps to reproduce the problem.
- ☐ Environmental details (e.g., operating system, browser, hardware).
- ☐ Stack traces, log files, or performance profiles.
- ☐ A clear description of the expected versus actual behaviour.
- ☐ Confirmation of whether the issue is intermittent or dependent on specific conditions.
- ☐ Other: _____

SQ6a *Optional: Is there another recurring gap not listed above, or any additional detail you would like to add to your answer?* *[Open Text Field]*

2.4 Coping Strategies and Heuristics (O2 / RQ1)

Purpose: Directly addresses **Research Objective 2 (O2)** and **Research Question 1 (RQ1)**. Examines what actions practitioners take first when steps are missing, and what practical rules of thumb they use to assign priority when a bug cannot be reproduced [12–14, 16].

SQ7 What is your primary immediate action when an assigned bug report lacks clear steps to reproduce? *[Single Choice, Required]*

- ☐ Request further clarification from the reporter and await a response.
- ☐ Consult a colleague or domain expert for contextual knowledge.
- ☐ Search the repository for similar past bugs or duplicates.
- ☐ Close the report immediately as Non-Reproducible or WorksForMe.

☐ Other: _____

SQ7a *Optional: Is your usual first step different from those listed above, or is there additional detail you would like to add to your answer? [Open Text Field]*

SQ8 **If you cannot reproduce a bug but still need to assign a priority, which cognitive heuristic or rule of thumb do you rely on most?** *[Single Choice, Required]*

- ☐ Compare the vague symptoms to recognised patterns from past bugs.
- ☐ Mentally simulate the code or scenario to evaluate the potential impact.
- ☐ Assign a default medium priority until more data emerges.
- ☐ Rely on the historical reliability and technical expertise of the reporter.
- ☐ Other: _____

SQ8a *Optional: Do you rely on a rule of thumb not listed above, or is there additional detail you would like to add to your answer? [Open Text Field]*

2.5 Organisational and Contextual Factors (O3 / RQ2)

Purpose: Directly addresses **Research Objective 3 (O3)** and **Research Question 2 (RQ2)**. Looks at how release deadlines, heavy workloads, backlog size, and management expectations push practitioners to make quick trade-offs during triage [17–20].

SQ9 **Which organisational constraints most heavily influence your triage decisions when evaluating a defect of unclear severity?** *[Select up to two options, Required]*

- ☐ Imminent release deadlines or deployment schedules.
- ☐ Current workload and the size of the team’s backlog.
- ☐ Management expectations or shifting business priorities.
- ☐ Conflicting priorities or disagreements across teams or stakeholders.
- ☐ Other: _____

SQ9a *Optional: How do these constraints change your final decision compared to a scenario with unlimited investigation time? [Open Text Field]*

2.6 Reflection and Tooling Improvements (O4)

Purpose: Supports **Research Objective 4 (O4)**. Gathers practitioner feedback on what tool features would help them handle uncertain bug reports better, and provides an open space for any additional thoughts [14, 19, 21].

SQ10 **Based on your experience, what tooling improvement would help you the most when dealing with incomplete or uncertain bug reports?** *[Single Choice, Required]*

- ☐ Better templates that force reporters to include steps and logs before submitting.
- ☐ Automated tools that capture the user’s environment and actions in a sandbox.
- ☐ Smarter search tools to easily find duplicate or related bug reports.
- ☐ Better ways to communicate directly with the reporter inside the tracking tool.
- ☐ Other: _____

- SQ10a** *Optional: Do you have another idea not listed above, or is there additional detail you would like to add to your answer? [Open Text Field]*
- SQ11** **Is there anything else about your decision-making process under uncertainty during bug triage that you would like to share?** *This is an open-ended space for any additional thoughts. [Open Text Field]*

References

- [1] J. W. Creswell and C. N. Poth, *Qualitative Inquiry and Research Design: Choosing Among Five Approaches*. SAGE Publications, 4th ed., 2018.
- [2] P. Runeson and M. Höst, “Guidelines for conducting and reporting case study research in software engineering,” *Empirical Software Engineering*, vol. 14, no. 2, pp. 131–164, 2009.
- [3] A. Bryman, *Social Research Methods*. Oxford University Press, 5th ed., 2016.
- [4] J. Anvik and G. C. Murphy, “Reducing the effort of bug report triage: Recommenders for development-oriented decisions,” *ACM Transactions on Software Engineering and Methodology (TOSEM)*, vol. 20, no. 3, pp. 1–35, 2011.
- [5] S. Mani, A. Sankaran, and R. Aralikkatte, “Deeptrriage: Exploring the effectiveness of deep learning for bug triaging,” in *Proceedings of the ACM India Joint International Conference on Data Science and Management of Data (CODS-COMAD ’19)*, pp. 171–179, ACM, 2019.
- [6] D. Bowes, T. Hall, and J. Petrić, “Software defect prediction: do different classifiers find the same defects?,” *Software Quality Journal*, vol. 26, no. 2, pp. 525–552, 2018.
- [7] N. Bettenburg, S. Just, A. Schröter, C. Weiss, R. Premraj, and T. Zimmermann, “What makes a good bug report?,” in *Proceedings of the 16th ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE ’08)*, pp. 308–318, ACM, 2008.
- [8] M. E. Joorabchi, M. Mirzaaghaei, and A. Mesbah, “Works for me! characterizing non-reproducible bug reports,” in *Proceedings of the 11th Working Conference on Mining Software Repositories (MSR ’14)*, pp. 62–71, ACM, 2014.
- [9] M. M. Rahman, F. Khomh, and M. Castelluccio, “Works for me! cannot reproduce – a large scale empirical study of non-reproducible bugs,” *Empirical Software Engineering*, vol. 27, no. 5, p. 111, 2022.
- [10] M. Begum and Y. Dittrich, “What happens in triage? an empirical study of bug triage in software product evolution,” Tech. Rep. TR-2024-210, IT University of Copenhagen, 2024.
- [11] D. Bertram, A. Voids, S. Greenberg, and R. Walker, “Communication, collaboration, and bugs: The social nature of issue tracking in small, colocated teams,” in *Proceedings of the 2010 ACM Conference on Computer Supported Cooperative Work (CSCW ’10)*, pp. 291–300, ACM, 2010.
- [12] G. A. Klein, *Sources of Power: How People Make Decisions*. MIT Press, 20th anniversary ed. ed., 2017.
- [13] R. Lipshitz, G. Klein, J. Orasanu, and E. Salas, “Taking stock of naturalistic decision making,” *Journal of Behavioral Decision Making*, vol. 14, no. 5, pp. 331–352, 2001.
- [14] R. Mohanani, I. Salman, B. Turhan, P. Rodríguez, and P. Ralph, “Cognitive biases in software engineering: A systematic mapping study,” *IEEE Transactions on Software Engineering*, vol. 46, no. 12, pp. 1318–1339, 2020.

- [15] S. Hove and B. Anda, “Experiences from conducting semi-structured interviews in empirical software engineering research,” in *11th IEEE International Software Metrics Symposium (METRICS '05)*, pp. 10–23, IEEE, 2005.
- [16] P. J. Guo, T. Zimmermann, N. Nagappan, and B. Murphy, “Characterizing and predicting which bugs get fixed: An empirical study of microsoft windows,” in *Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering (ICSE '10)*, pp. 495–504, ACM, 2010.
- [17] M. Kuuttila, M. M”antyl”a, U. Farooq, and M. Claes, “Time pressure in software engineering: A systematic review,” *Information and Software Technology*, vol. 121, p. 106257, 2020.
- [18] L. Erlikh, “Leveraging legacy system dollars for e-business,” *IT Professional*, vol. 2, no. 3, pp. 17–23, 2000.
- [19] O. Baysal, R. Holmes, and M. W. Godfrey, “No issue left behind: Reducing information overload in issue tracking,” in *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE '14)*, pp. 666–677, ACM, 2014.
- [20] H. Jahanshahi, M. Cevik, K. Mousavi, and A. Bařar, “ADPTriage: Approximate dynamic programming for bug triage under uncertainty,” *IEEE Transactions on Software Engineering*, vol. 49, no. 10, pp. 4594–4609, 2023.
- [21] U. B. Torun, M. T. Demircan, M. F. G”on, and E. T”uz”un, “Past, present, and future of bug tracking in the generative ai era,” *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 2026. Just Accepted.